Guía 2 de 3 — IA local con Ollama: prompts, apply y la regla de oro

DPL1046 · Clase 8 · Para repasar en casa, a tu ritmo.

Una IA que corre **en tu propio computador** funciona como cualquier otra API: le mandas un texto y te devuelve otro. Esta guía repasa cómo hablar con ella desde Python, cómo escribir un buen prompt y cómo usarla sin que te invente números.

> Requisitos (una sola vez): tener **Ollama instalado y abierto** y haber ejecutado `ollama pull llama3.2` (≈2 GB; hazlo con buen internet). Si no lo tienes, los scripts del curso funcionan en **modo simulado** y puedes seguir igual.

---

1. Una IA es solo otra API

En la Clase 4 vimos la API como un **garzón**: le llevas un pedido y te trae la respuesta. Una IA local funciona **igual**: tu script le manda un texto (el **prompt**) y ella responde otro texto. La única diferencia es la dirección: en vez de un sitio en internet, es `localhost`, **tu propio computador**. Por eso **nada de lo que le envías sale a internet**.

```
import requests

URL = "http://localhost:11434/api/generate"
MODELO = "llama3.2"
pregunta = "En una sola frase, ¿qué es un DataFrame de pandas?"

try:
    respuesta = requests.post(
        URL,
        json={"model": MODELO, "prompt": pregunta, "stream": False},
        timeout=120,
    )
    respuesta.raise_for_status()
    texto = respuesta.json()["response"]
    print("Pregunta:", pregunta)
    print("Respuesta de la IA:", texto.strip())
except requests.exceptions.ConnectionError:
    print("No encontré Ollama. ¿Está abierto?")
except requests.exceptions.HTTPError:
    print("Ollama respondió con error. ¿Descargaste el modelo? ollama pull
llama3.2")
```

- `localhost` significa “este mismo computador”; `11434` es el **puerto** donde Ollama escucha (como el número de oficina de un edificio).

- Antes usábamos `requests.get` (pedir datos); ahora `requests.post`, que **envía** información.
- `json={...}` es lo que mandamos: el `model`, el `prompt` y `stream: False` ("dame la respuesta completa, no por pedazos").
- `timeout=120`: una IA local puede demorar; damos dos minutos. **La primera llamada tarda más** porque el modelo se está cargando.
- La respuesta es JSON: el texto de la IA está en la clave `response`; `strip()` quita espacios y saltos de línea sobrantes.
- `ConnectionError` = Ollama **no está abierto**; `HTTPError` = normalmente **falta descargar el modelo**.

---

2. Un buen prompt

Un prompt es como **darle instrucciones a un practicante**. Si le dices "revisa esto", hará lo que se le ocurra. Si le dices "eres analista, clasifica esta reseña como positiva o negativa y responde **solo una palabra**", el resultado es predecible.

Ingrediente	Qué es	Ejemplo
Rol	Quién eres	"Eres analista de datos."
Tarea	Qué hacer	"Clasifica esta reseña como positiva o negativa."
Formato	Cómo responder	"Responde SOLO con una palabra."
Datos	Sobre qué	"Reseña:" + texto

---

3. apply y lambda: repetir una función en cada fila

`apply` es el `for` de pandas: "toma esta función y aplícala **a cada fila**". Como una línea de producción: cada pieza pasa por la misma máquina. `lambda` es una **mini función sin nombre**, de una línea.

```
# Estas dos cosas son lo mismo:
def sumar_uno(x):
    return x + 1

sumar_uno = lambda x: x + 1
```

Clasificar reseñas

```
def clasificar(texto, puntaje):
    """Devuelve 'positiva' o 'negativa' para una reseña."""
    prompt = (
        "Clasifica esta reseña de un cliente como positiva o negativa. "
        "Responde SOLO con una palabra: positiva o negativa.\n"
        "Reseña: " + texto
    )
    try:
        respuesta = requests.post(
            URL, json={"model": MODELO, "prompt": prompt, "stream": False},
timeout=120
        )
        respuesta.raise_for_status()
        palabra = respuesta.json()["response"].strip().lower()
        return "negativa" if "neg" in palabra else "positiva"
    except requests.exceptions.RequestException:
        return "positiva (simulado)" if puntaje >= 4 else "negativa (simulado)"

reviews = pd.read_csv("data/raw/olist_order_reviews_dataset.csv")
con_texto = reviews.dropna(subset=["review_comment_message"])
muestra = con_texto[["review_score", "review_comment_message"]].head(8).copy()

muestra["sentimiento"] = muestra.apply(
    lambda fila: clasificar(fila["review_comment_message"], fila["review_score"]),
axis=1
)
print(muestra["sentimiento"].value_counts())
```

- Los textos entre comillas, uno bajo otro dentro de un paréntesis, se **pegan automáticamente**; `\n` es un salto de línea.
- `.strip().lower()` quita espacios y pasa a minúsculas, para comparar sin sorpresas.
- `"negativa" if "neg" in palabra else "positiva"` es una **decisión en una línea**. Se busca `"neg"` y no la palabra exacta porque la IA a veces agrega puntos o espacios.
- El `except` devuelve una **regla simple** marcada como "(simulado)": así, quien lea la tabla sabe que **no fue la IA**. Un buen pipeline siempre deja claro de dónde vino cada dato.
- `apply(..., axis=1)` recorre **fila por fila** y llama a `clasificar` con el comentario y el puntaje: son 8 llamadas a la IA (por eso solo usamos 8 reseñas: cada una tarda).

---

4. La regla de oro: pandas calcula, la IA explica

La IA es como un **locutor**: lee muy bien lo que le pongan en el papel, pero **no verifica las cuentas**. Puede inventar un número con total seguridad. Entonces:

> **Los números los calcula siempre pandas. La IA solo redacta con esos números. Nunca al revés.**

```
# 1) Pandas calcula
promedio = round(dias.mean(), 1)
lentos = round((dias > 20).mean() * 100, 1)

# 2) La IA solo redacta con esos números
prompt = (
    "Eres analista de datos. Escribe un resumen ejecutivo de 3 líneas en español,
"
    "usando SOLO estos datos, sin inventar otros:\n"
    f"- Promedio de días de entrega: {promedio}\n"
    f"- Entregas lentas (más de 20 días): {lentos}%\n"
)
```

- `(dias > 20).mean() * 100`: `dias > 20` da verdadero/falso por fila; el `.mean()` de verdaderos y falsos es la **proporción de verdaderos** (True vale 1, False vale 0); por 100 es el **porcentaje**.
- La `f` antes de las comillas crea un **f-string**: lo que va entre `{ }` se reemplaza por el valor de la variable.
- La instrucción clave es **"usando SOLO estos datos, sin inventar otros"**.
- Después de ejecutar, **compara los números del resumen con los de pandas**: la IA solo puede usar lo que le diste.

---

5. Antes de enviar datos a una IA: 4 preguntas

#	Pregunta	Si la respuesta es "sí"
1	¿Hay datos personales (RUT, nombre, correo)?	Protégelos (hash/mask) o usa IA local
2	¿Hay claves o secretos en el texto?	**Nunca** los envíes
3	¿Los datos son confidenciales de tu empresa?	IA local o pide autorización
4	¿Revisé la respuesta antes de usarla?	Si no, no la uses

Por eso usamos IA local: es la forma más simple de cuidar la privacidad. Y registrar tus prompts con `logging` sirve de auditoría.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
"No encontré Ollama"	La aplicación no está abierta	Ábrela y vuelve a ejecutar
`HTTPError` / modelo no encontrado	No descargaste el modelo	`ollama pull llama3.2` en la terminal
El script parece "colgado"	La primera llamada carga el modelo	Espera; no cierres la terminal
La IA responde algo distinto a una palabra	El prompt no fue claro	Agrega "Responde SOLO con…" y revisa con `"neg" in palabra`
Los números del resumen no calzan	La IA inventó datos	Aplica la regla de oro: calcula con pandas y pásale solo esos números

Practica tú

1. ¿Qué cambias para que la IA clasifique 12 reseñas en vez de 8?
2. Escribe el prompt de un resumen que incluya también el **mínimo** de días de entrega.
3. ¿Por qué marcamos "(simulado)" en el plan B?
4. ¿Qué pregunta de las 4 aplica si el texto contiene correos de clientes?

▼ **Ver respuestas**

1. `.head(12)` en vez de `.head(8)` (tardará más).
2. Calcula `minimo = dias.min()` con pandas y agrega la línea `f"- Mínimo de días de entrega: {minimo}\n"` al prompt.
3. Para ser honestos: esos valores vienen de una regla, no de la IA.
4. La pregunta 1: hay datos personales; se protegen (hash/mask) o se usa IA local.

Para llevarte

- Una IA local es **otra API**: `requests.post` a `localhost`; nada sale de tu computador.
- Buen prompt = **rol + tarea + formato + datos**.
- `apply` + `lambda` aplican una función a cada fila.
- **Pandas calcula, la IA explica.** Siempre revisa la respuesta.